



**SAPIENZA**  
UNIVERSITÀ DI ROMA

**Sapienza University of Rome**

Dipartimento di ingegneria informatica, automatica e gestionale "Antonio  
Ruberti"

Corso di Master Degree in Artificial Intelligence and Robotics

# **Multi-agent Decision Support System for Space Mission Operations**

Thesis Advisor  
**Prof. Danilo Comminiello**

Candidate  
**Antonio Lisa Lattanzio**  
**2154208**

**Academic Year MMXXV-MMXXVI**



## Abstract

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Keywords: lorem, ipsum, dolor, sit, amet, consectetur, adipisicing, elit, sed, do.



# Contents

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                                                | <b>iv</b> |
| <b>List of Tables</b>                                                                 | <b>v</b>  |
| 0.1 The Evolution of Natural Language Processing and Decision Support Systems . . . . | 1         |
| 0.1.1 Early Symbolic Approaches . . . . .                                             | 1         |
| 0.1.2 Early Neural Approaches . . . . .                                               | 3         |
| 0.1.3 From Static Embeddings to Sequential Modeling . . . . .                         | 5         |
| 0.1.4 The Transformer Revolution . . . . .                                            | 6         |
| 0.1.5 Scaling Laws and Emergent Abilities . . . . .                                   | 7         |
| 0.1.6 Foundation Models and General-Purpose AI . . . . .                              | 7         |
| 0.1.7 Alignment and Instruction Tuning . . . . .                                      | 8         |
| 0.1.8 The Reliability Gap: Hallucinations and Uncertainty . . . . .                   | 8         |
| 0.1.9 Computational Burden and Compression Strategies for LLMs . . . . .              | 9         |
| 0.1.10 Reasoning Frameworks for Decision Support . . . . .                            | 10        |
| 0.1.11 Long-Context Modeling and Context Management . . . . .                         | 11        |
| 0.2 Agentic Large Language Models . . . . .                                           | 11        |
| 0.3 Evolution of Decision Support Systems . . . . .                                   | 11        |
| <b>Bibliography</b>                                                                   | <b>12</b> |

# List of Figures

# List of Tables

|   |                                                                                                       |   |
|---|-------------------------------------------------------------------------------------------------------|---|
| 1 | Bag-of-Words representation of two syntactically different but lexically identical sentences. . . . . | 1 |
|---|-------------------------------------------------------------------------------------------------------|---|

## 0.1 The Evolution of Natural Language Processing and Decision Support Systems

The field of Natural Language Processing (NLP) has experienced a remarkable evolution over the past decades. Early approaches relied on manually engineered representations and statistical models, where language was treated as a collection of symbolic features rather than as a structured and contextual system. Advances in machine learning progressively shifted the focus toward distributed representations and neural architectures capable of learning linguistic patterns directly from data. This progression ultimately culminated in the Transformer architecture, which forms the foundation of modern Large Language Models (LLMs). Understanding this historical evolution is essential for appreciating the capabilities and limitations of current AI-based Decision Support Systems (DSS).

The development of NLP can be broadly divided into three major phases. The first phase is characterized by sparse statistical representations such as Bag-of-Words and n-gram language models. The second phase introduced recurrent neural architectures capable of modeling sequential dependencies. Finally, the third phase replaced recurrence with attention mechanisms, enabling scalable contextual reasoning over long sequences.

### 0.1.1 Early Symbolic Approaches

Early NLP systems were primarily based on symbolic representations and sparse statistical features. Rather than learning semantic representations from data, documents were encoded through manually designed features derived from word occurrences. One of the most influential representations was the *Bag-of-Words* (BoW) model [26, 32]. In this representation, each document is transformed into a vector containing the frequency of every word in a predefined vocabulary, while completely disregarding grammar, syntax, and word order.

For example, consider the following two sentences:

"The doctor treats the patient."

"The patient treats the doctor."

Although the semantic meaning of the two sentences is clearly different, the Bag-of-Words representation assigns them nearly identical vectors because they contain exactly the same set of words. The only retained information is the frequency of each term, whereas all structural information is discarded. Consequently, language is reduced to an unordered multiset of independent lexical features.

This can be explicitly observed in their Bag-of-Words representations, as shown in Table 1.

| Term    | Sentence 1 | Sentence 2 |
|---------|------------|------------|
| the     | 2          | 2          |
| doctor  | 1          | 1          |
| patient | 1          | 1          |
| treats  | 1          | 1          |

**Table 1:** Bag-of-Words representation of two syntactically different but lexically identical sentences.

Although the semantic meaning of the two sentences is clearly different, the Bag-of-Words representation assigns them nearly identical vectors because they contain exactly the same set



of words. The only retained information is the frequency of each term, whereas all structural information is discarded. Consequently, language is reduced to an unordered multiset of independent lexical features.

Despite this limitation, Bag-of-Words proved remarkably effective for several tasks, including document classification, spam detection, topic categorization, and information retrieval. Its simplicity, low computational cost, and compatibility with classical machine learning algorithms such as Naïve Bayes and Support Vector Machines [26] contributed to its widespread adoption.

However, the independence assumption underlying BoW prevents the model from capturing compositional meaning or contextual relationships between words. Expressions such as "artificial intelligence", "credit card", or "New York" are decomposed into independent terms, ignoring the fact that their meaning emerges from the combination of multiple words rather than from each individual token.

The statistical foundations of these early methods were strongly influenced by the *distributional hypothesis*, originally proposed by Harris [16], which states that words occurring in similar linguistic contexts tend to exhibit similar semantic properties. This principle later became one of the cornerstones of statistical NLP and distributional semantics.

Motivated by the need to incorporate sequential information, researchers introduced *n-gram language models* [11]. Instead of treating words as independent observations, n-gram models estimate the probability of a word by conditioning only on the previous  $n - 1$  tokens according to the Markov assumption:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-(n-1)}, \dots, w_{t-1}) \quad (1)$$

Depending on the value of  $n$ , different language models can be obtained. A unigram model ( $n = 1$ ) assumes complete independence between words and is therefore equivalent to the Bag-of-Words assumption. A bigram model predicts each word using only the immediately preceding token, whereas a trigram model considers the previous two words.

For instance, given the sentence

"The patient needs medical assistance."

the extracted n-grams are:

- Unigrams: {The, patient, needs, medical, assistance}
- Bigrams: {The patient, patient needs, needs medical, medical assistance}
- Trigrams: {The patient needs, patient needs medical, needs medical assistance}

Unlike Bag-of-Words, these representations preserve a limited notion of local word order, allowing statistical models to learn common expressions, collocations, and short syntactic structures.

Nevertheless, n-gram models remain fundamentally constrained by their fixed context window. Long-range dependencies cannot be modeled because only the previous  $n - 1$  tokens are considered. For example, in the sentence

"The physician who examined the patient yesterday **prescribes** the medication."

the relationship between the subject “physician” and the verb “prescribes” spans several intermediate words. A small bigram or trigram model cannot capture such dependencies effectively.

A second limitation concerns scalability. As the value of  $n$  increases, the number of possible word combinations grows exponentially. Consequently, even very large corpora contain only a small fraction of all possible  $n$ -grams, leading to severe data sparsity problems.

These shortcomings motivated the transition toward neural language models capable of learning continuous distributed representations instead of relying exclusively on explicit frequency counts.

### 0.1.2 Early Neural Approaches

To overcome the limitations of  $n$ -gram models, early neural approaches introduced the idea of learning distributed representations over sequences. Simple Recurrent Networks (SRNs) by Elman [10] established a foundational architecture for processing sequential data and are considered a precursor to modern Recurrent Neural Networks (RNNs).

The training of neural networks, including recurrent architectures, is based on backpropagation [34]. In the recurrent setting, this is extended to Backpropagation Through Time (BPTT) [14], which enables gradient-based optimization over temporal dependencies by unrolling the network across time.

Unlike feedforward models, RNNs process input tokens sequentially while maintaining a hidden state that acts as a memory of past information. A standard formulation of an RNN is given by:

$$h_t = f(W_h h_{t-1} + W_x x_t + b_h) \quad (2)$$

$$y_t = W_y h_t + b_y \quad (3)$$

This formulation allows, in principle, the modeling of arbitrarily long dependencies through the recurrent hidden state. However, training RNNs in practice is challenging due to the vanishing and exploding gradient problems, which limit their ability to preserve information over long sequences.

To understand this issue, consider a simplified recurrent formulation:

$$h_t = \phi(W h_{t-1})$$

Under BPTT, the gradient of the loss with respect to a hidden state can be expressed as a product of Jacobians across time:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

Taking the norm yields:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \approx \prod_{k=t}^{T-1} \left\| \frac{\partial h_{k+1}}{\partial h_k} \right\|$$

This shows that the gradient magnitude is determined by repeated multiplication of Jacobian matrices across time steps. Depending on their spectral properties, this product can either grow or decay exponentially, leading respectively to exploding or vanishing gradients.

This fundamental limitation significantly hinders the ability of vanilla RNNs to learn long-term dependencies, and motivates the development of more advanced recurrent architectures designed to improve gradient flow over long sequences.

To address these issues, more advanced architectures were proposed, most notably Long Short-Term Memory (LSTM) networks [17]. LSTMs introduce an explicit memory cell  $c_t$  designed to preserve long-term information, together with a set of gating mechanisms that regulate information flow. The core idea is to decouple long-term memory from short-term hidden representations.

The LSTM update equations are defined as:

$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \\ \tilde{C}_t &= \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \\ C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t \\ H_t &= O_t \odot \tanh(C_t) \end{aligned}$$

The key property of this formulation is the additive update of the cell state  $c_t$ , which contrasts with the purely multiplicative transformations of standard RNNs. This structure allows gradients to flow more effectively through time, since the derivative of  $c_t$  with respect to  $c_{t-1}$  is modulated by the forget gate  $f_t$ , which can learn to preserve information when needed.

As a result, LSTMs significantly mitigate the vanishing gradient problem and enable learning of long-range dependencies that are difficult to capture with vanilla RNNs. This makes them particularly effective in sequence modeling tasks such as machine translation, speech recognition, and sequence labeling.

A further simplification of this architecture led to Gated Recurrent Units (GRUs), which combine the forget and input gates into a single update mechanism. GRUs maintain similar performance to LSTMs in many tasks while reducing computational complexity and the number of parameters.

The GRU formulation is defined as:

$$\begin{aligned} Z_t &= \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z) \\ R_t &= \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r) \\ \tilde{H}_t &= \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h) \\ H_t &= (1 - Z_t) \odot H_{t-1} + Z_t \odot \tilde{H}_t \end{aligned}$$

Both architectures represent a critical improvement over vanilla RNNs by explicitly addressing the instability of long-term temporal dependencies.

Despite their success, RNN-based architectures still process sequences sequentially, limiting parallelization during training and inference. This inherent inefficiency, combined with residual difficulties in modeling extremely long dependencies, eventually motivated the development of attention-based models and Transformers, which replaced recurrence with global context modeling.

This progression from BoW and n-gram models to RNNs and gated variants marks a fundamental shift in NLP: from static, order-agnostic representations toward dynamic, stateful systems capable of modeling language as a temporal process. This evolution forms the conceptual bridge toward modern contextual embedding methods and Transformer-based architectures.

### 0.1.3 From Static Embeddings to Sequential Modeling

The first fundamental advancement in how machines represent language was the introduction of *Word Embeddings*. The most influential work in this area is **Word2Vec**, proposed by Mikolov et al. [27]. This model demonstrated that words could be mapped into a high-dimensional vector space where semantic relationships are preserved as geometric distances. While Word2Vec enabled early NLP tasks to capture basic conceptual analogies, these embeddings remained **static**: a single vector was assigned to a word regardless of its context (e.g., "bank" in a financial vs. geographical sense). Moreover, Word2Vec does not model word order or syntactic structure, since embeddings are learned independently of the surrounding sentence. As a consequence, it cannot capture long-range dependencies or contextual nuances that are essential in many decision-support scenarios. The representation is therefore powerful at the lexical level but fundamentally limited for tasks requiring a deeper understanding of sequence dynamics.

Shortly after, Pennington et al. introduced **GloVe** (Global Vectors for Word Representation) [30]. GloVe refined the embedding approach by combining the advantages of global matrix factorization—leveraging statistics from the entire corpus—with local context window methods. While both Word2Vec and GloVe were revolutionary, they shared a fundamental limitation: they produced **static** embeddings. A single vector was assigned to a word regardless of its context (e.g., the word "bank" would have the same representation in a financial vs. a geographical sense). Furthermore, these models do not account for word order or complex syntactic variations.

To address the issue of polysemy and contextual nuance, the field moved toward **Deep Contextualized Word Representations**. A landmark in this transition was **ELMo** (Embeddings from Language Models), proposed by Peters et al. [31]. Unlike static embeddings, ELMo's vectors are learned functions of the internal states of a deep bidirectional language model (biLM). By exposing the deep internal layers of a pre-trained network, ELMo allowed downstream models to capture how word meanings vary across different linguistic contexts. This represented a shift from fixed lookup tables to dynamic, context-aware representations, significantly improving performance on tasks such as sentiment analysis and textual entailment.

Parallel to the development of embeddings, the state-of-the-art for processing sequences was represented by Recurrent Neural Networks (RNNs) [34]. However, standard RNNs were notoriously difficult to train on long sequences due to the vanishing gradient problem, as formally analyzed by Bengio et al. [3]. To overcome these limitations, the Long Short-Term Memory (LSTM) architecture was introduced [17]. By employing a gating mechanism to regulate information flow, LSTMs became a dominant architecture for sequence modeling across many NLP tasks, enabling models to maintain a form of memory over long input sequences.

#### 0.1.4 The Transformer Revolution

The definitive breakthrough occurred with the publication of "*Attention Is All You Need*" [37]. Vaswani et al. proposed a novel architecture entirely based on the **Self-Attention mechanism**, dispensing with recurrence and convolutions. The core innovation lies in the model's ability to weigh the significance of different parts of the input data independently of their distance in the sequence. Mathematically, the Scaled Dot-Product Attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (4)$$

where  $Q$ ,  $K$ , and  $V$  represent the Query, Key, and Value matrices, and  $d_k$  is the dimension of the keys. The scaling factor  $\sqrt{d_k}$  plays a crucial role in stabilizing the optimization process. Without this normalization term, the dot products between high-dimensional query and key vectors could grow excessively large, causing the softmax function to produce extremely peaked probability distributions and resulting in vanishing gradients during training. The scaling operation therefore contributes to more stable learning dynamics and improved convergence properties.

Conceptually, the attention mechanism can be interpreted as a weighted aggregation process. Each query vector determines how much importance should be assigned to all available key-value pairs, enabling the model to selectively focus on the most relevant contextual information for a given token. This selective processing capability constitutes one of the fundamental reasons behind the remarkable effectiveness of Transformer-based architectures. This mechanism allows the model to build a global understanding of the input, enabling the emergence of "contextual intelligence" where word representations change dynamically based on the surrounding text. In the context of NLP, this marked the move from simple pattern recognition to sophisticated reasoning, as models could now integrate informations coming from diverse and distant data points. A key extension introduced in the original Transformer architecture is the multi-head self-attention mechanism. Instead of computing a single attention map, the model projects the input into multiple sets of query, key, and value spaces and applies the attention function in parallel across these sets.

Formally, multi-head attention is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (5)$$

where

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (6)$$

and  $W_i^Q$ ,  $W_i^K$ ,  $W_i^V$ , and  $W^O$  are learnable projection matrices. This formulation enables each attention head to specialize in capturing different forms of dependencies within the data, thereby enriching the overall representation learned by the model.

The outputs from these attention heads are then concatenated and linearly transformed, allowing the model to jointly attend to diverse types of relationships in the input sequence, such as syntactic and semantic patterns simultaneously. By combining multiple attention heads, the network can capture richer representations than a single attention head would allow, without increasing the overall model depth.

Another important practical advantage of Transformer architectures is their massive paralleliza-

tion potential. Unlike recurrent networks, which process tokens sequentially, each attention layer in a Transformer can compute interactions between all positions in the input sequence simultaneously. This property stems directly from the self-attention formulation and makes the architecture highly amenable to batched matrix operations on modern hardware accelerators such as GPUs and TPUs. As a result, Transformers can be trained more efficiently on large corpora, enabling the scale-up that underpins modern large language models.

### 0.1.5 Scaling Laws and Emergent Abilities

The evolution culminated in the transition from task-specific models to *Large Language Models* (LLMs). This shift was driven by the discovery of scaling laws [22], which suggested that model performance, and consequently capabilities on NLP tasks, improve with increases in parameter count, dataset size, and compute, which can indirectly improve the performance of systems that leverage NLP components, including DSS applications.

Furthermore, the introduction of the Pre-training and Fine-tuning paradigm, popularized by BERT [9] and the GPT series [33], enabled these systems to internalize vast amounts of world knowledge. This trajectory led to the discovery of *emergent abilities*: capabilities not explicitly present in smaller models, such as in-context learning and multi-step logical reasoning [39]. For modern Decision Support Systems, this represents a transition from "engines that process text" to systems that can generate knowledge-informed recommendations, while human oversight remains necessary.

### 0.1.6 Foundation Models and General-Purpose AI

The scalability of the Transformer architecture, coupled with the empirical discovery of scaling laws, has catalyzed a paradigm shift in artificial intelligence: the emergence of **Foundation Models**. As defined by Bommasani et al. [6], a foundation model is any model trained on a vast amount of data, typically using self-supervision at scale, that can be adapted (e.g., fine-tuned) to a wide range of downstream tasks. This represents a fundamental departure from the traditional AI development lifecycle, where models were painstakingly engineered for single, narrow objectives. Instead, foundation models serve as a versatile base, encoding broad representations of language, logic, and world knowledge that can be repurposed across diverse domains, including the sophisticated requirements of Decision Support Systems.

A pivotal demonstration of this potential was provided by Brown et al. [7] with the introduction of GPT-3. This work highlighted that when language models are scaled to billions of parameters, they begin to exhibit *in-context learning* capabilities. Unlike previous approaches that required gradient-based fine-tuning to perform a new task, GPT-3 demonstrated the ability to adapt to specific instructions simply by observing a few examples provided within the input prompt (few-shot learning). This shift fundamentally changed the interaction between users and systems, moving from "training" to "prompting" as the primary mechanism for directing model behavior.

However, the rapid expansion of model size brought to light significant challenges regarding computational efficiency. The industry trend initially focused on increasing parameter counts indiscriminately; this was challenged by Hoffmann et al. [18] in their study on *Chinchilla*. The authors demonstrated that the performance of a language model is not solely dependent on its parameter

count, but rather on an optimal balance between model size and the volume of training data. Their findings established new scaling laws, suggesting that most contemporary models were significantly under-trained. This insight provided a roadmap for developing "compute-optimal" models, enabling superior performance with more efficient resource allocation.

### 0.1.7 Alignment and Instruction Tuning

The transition from raw scale to functional utility in Decision Support Systems was enabled by the development of *Alignment* techniques. While base LLMs possess extensive statistical information derived from their training corpora, they often fail to follow specific user intents or produce helpful, safe, and concise reasoning. To bridge this gap, **Instruction Tuning** was introduced, most notably through the work on **InstructGPT** by Ouyang et al. [29]. This process involves fine-tuning the model on a dataset of human-written prompt-response pairs, shifting the objective from simple next-token prediction to "intent fulfillment."

A critical component of this alignment is **Reinforcement Learning from Human Feedback (RLHF)**. This methodology, popularized by Christiano et al. [8] and later refined for LLMs [36], allows models to optimize their outputs based on human preferences. This technique helps to guide the model towards outputs that better reflect the human preferences and alignment objectives, though hallucinations and biases can still occur.

### 0.1.8 The Reliability Gap: Hallucinations and Uncertainty

The primary obstacle to the integration of LLMs in critical Decision Support Systems (DSS) is the phenomenon of **hallucinations**, defined as the generation of factually incorrect or ungrounded statements despite high linguistic fluency [20]. This stems from the underlying training objective of next-token log-likelihood, which prioritizes plausible linguistic continuation over factual verification. In a DSS context, **extrinsic hallucinations**, where the output cannot be verified from provided sources or real-world knowledge, are particularly hazardous, as the model may provide a confident but false justification for a recommendation [2].

Beyond explicit errors, a critical risk is the **lack of self-awareness regarding uncertainty**. State-of-the-art models are often poorly calibrated, meaning their internal probability estimates do not align with their actual correctness [21].

Recent surveys [1] identify a broad taxonomy of **hallucination detection methods**, designed to assess the factuality and faithfulness of LLM outputs before they are used in high-stakes Decision Support Systems.

- **Retrieval-based methods** detect hallucinations by comparing generated statements against external knowledge sources or retrieved evidence. These techniques, often implemented through RAG pipelines [23], are particularly effective for identifying *extrinsic hallucinations* by checking whether claims are supported by authoritative documents.
- **Uncertainty-based approaches** estimate unreliability through internal model signals such as token-level probabilities, entropy, attention stability, or structured uncertainty propagation. This line of work directly addresses the calibration issues highlighted in [21], offering a principled way to detect when a model is uncertain despite fluent output.

- **Embedding-based methods** measure semantic alignment between inputs, outputs, and reference sources within a shared embedding space. Low semantic similarity indicates drift from the provided context or task grounding.
- **Learning-based detectors**, including supervised, weakly supervised, and agent-based systems, train dedicated models to classify hallucinated versus factual outputs. These techniques capture complex error patterns that cannot be identified through simple probability thresholds or similarity metrics.
- **Self-consistency-based techniques** generate multiple answers or paraphrased queries to assess stability across model outputs. Inconsistencies in meaning or reasoning serve as indicators of potential hallucinations, making these methods valuable when external verification is unavailable.

Overall, this taxonomy underscores that mitigating hallucinations involves not only filtering incorrect content but also understanding where and why LLM reasoning becomes unreliable, which is essential for safe integration into critical DSS.

Recent work suggests that some failure modes traditionally framed as hallucinations may stem not only from generative uncertainty but also from structural challenges in long-context evidence aggregation [5]. Studies on retrieval in extended contexts show that the size and distribution of relevant information influence an LLM’s ability to locate and integrate the correct evidence. When small but crucial “gold” contexts are embedded within much longer distractor passages, models exhibit reduced aggregation accuracy and increased positional sensitivity. These effects are particularly relevant for retrieval-augmented and agentic systems, where evidence sources often vary considerably in length. In such settings, short but important documents may be underutilized, potentially leading the model to produce unsupported outputs even when the necessary information is included in the input. These findings suggest that hallucination-related risks arise not only from calibration issues or insufficient grounding but also from context-size asymmetry and the design of evidence-aggregation pipelines.

### 0.1.9 Computational Burden and Compression Strategies for LLMs

Even though LLMs are extremely useful they also are computationally expensive, the development of new models has been characterized by an increase of the parameters size that grown to hundreds of billions of parameters in the last years, with hardware that are getting more and more powerful and optimized to keep up with the evolution of this technology [24]. The main methods to reduce the cost are applicable during the fine-tuning stage or the inference task:

- **Parameter-efficient fine-tuning (PEFT)**: refers to the process of adjusting the parameters of a pre-trained large model to adapt it to a specific task or domain while minimizing the number of additional parameters introduced or computational resources required [15]. One of their main methods is LoRA [19] which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, reducing the number of trainable parameters.
- **Parameters quantization**: focuses on reducing the size of the parameters to usually 4 or 8 bits like Smoothquant [41], interesting methods are also GPTQ [12], which quantizes weights



block-wise and iteratively corrects quantization errors using Hessian information, and AWQ, which focuses on reducing the error by preserving the most important weights [25].

- **Quantization Aware Fine Tuning:** QA-Lora represents a hybrid approach between PEFT and quantization, where the model is fine-tuned while accounting for quantization constraints [42].
- **Pruning:** focus on lightening the model size by removing the connections or neurons that are less important [13, 35]

Compression techniques inevitably involve trade-offs between efficiency and performance, as aggressive quantization, pruning, or parameter-efficient fine-tuning may degrade model accuracy or require additional implementation effort, and task-specific performance variability must be considered [12, 13, 19, 25].

### 0.1.10 Reasoning Frameworks for Decision Support

Beyond architectural and alignment improvements, the operational use of LLMs in decision-making has been revolutionized by specialized prompting and reasoning frameworks. The introduction of **Chain-of-Thought (CoT)** prompting [40] demonstrated that forcing a model to generate intermediate reasoning steps significantly enhances performance on complex logical tasks. While CoT provides a "slow thinking" approach essential for auditing a DSS, relying on a single greedy-decoded trajectory can lead to error propagation. To mitigate this, **Self-Consistency** [38] introduces a decoding strategy that samples multiple reasoning paths and selects the most frequent marginal result, leveraging the intuition that complex problems often admit multiple paths to a unique correct answer.

Further advancing non-linear reasoning, the **Tree of Thoughts (ToT)** [43] and **Graph of Thoughts (GoT)** [4] frameworks enable models to explore multiple solution branches or model dependencies as directed graphs. These structures allow for human-like problem solving, such as combining intermediate solutions or revising reasoning paths based on "thought volume" metrics. When coupled with **Retrieval-Augmented Generation (RAG)** [23] and acting frameworks like **ReAct** [44], LLMs shift from static predictors to interactive reasoning systems capable of querying external databases and updating internal states based on real-time observations.

The state-of-the-art in this domain has recently moved toward *adaptive, multi-agent architectures* that operationalize these reasoning frameworks within high-stakes environments. A primary example is the work of Ögdü et al. [28], who developed a multi-layered, adaptive Clinical Decision Support System (CDSS). Unlike traditional static models, this architecture integrates biomedical RAG with real-time web intelligence through a collaborative framework of autonomous agents. The system introduces several key innovations:

- **Timeliness and Evidence Freshness:** By incorporating a dedicated web-intelligence agent, the system mitigates the "knowledge cutoff" problem of static LLMs, ensuring recommendations are grounded in the latest clinical guidelines.
- **Confidence-Aware Optimization:** The system utilizes an adaptive feedback loop that triggers re-querying and hypothesis refinement whenever internal confidence scores fall below a defined threshold ( $\tau = 0.65$ ), effectively automating the iterative reasoning seen in ToT/GoT frameworks.

- **Domain-Specific Grounding:** To ensure precision in medical contexts, the system employs a specialized BioBERT-based vector database for processing complex clinical jargon.

Validated on the MedQA and PubMedQA datasets—achieving accuracies of 94% and 88% respectively, this approach demonstrates how the integration of multi-source data fusion and adaptive optimization loops can overcome the reliability barriers of standard LLM-based reasoning.

#### **0.1.11 Long-Context Modeling and Context Management**

### **0.2 Agentic Large Language Models**

### **0.3 Evolution of Decision Support Systems**

# Bibliography

- [1] A. Alansari and H. Luqman. Large language models hallucination: A comprehensive survey. *arXiv preprint arXiv:2510.06265*, 2025.
- [2] D. Anh-Hoang, V. Tran, and L.-M. Nguyen. Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior. *Frontiers in Artificial Intelligence*, Volume 8 - 2025, 2025.
- [3] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.
- [4] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17682–17690, 2024.
- [5] O. Bianchi, M. J. Koretsky, M. Willey, C. X. Alvarado, T. Nayak, A. Asija, N. Kuznetsov, M. A. Nalls, F. Faghri, and D. Khoshnab. Hidden in the haystack: Smaller needles are more difficult for llms to find. *arXiv preprint arXiv:2505.18148*, 2025.
- [6] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- [7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [8] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [10] J. L. Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.
- [11] G. A. Fink. n-gram models. In *Markov Models for Pattern Recognition: From Theory to Applications*, pages 107–127. Springer, 2014.

- 
- [12] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
  - [13] M. Gordon, K. Duh, and N. Andrews. Compressing bert: Studying the effects of weight pruning on transfer learning. In *Proceedings of the 5th Workshop on Representation Learning for NLP*, pages 143–155, 2020.
  - [14] R. Grosse. Lecture 15: Exploding and vanishing gradients. *University of Toronto Computer Science*, 2017.
  - [15] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*, 2024.
  - [16] Z. S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.
  - [17] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
  - [18] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.
  - [19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
  - [20] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.
  - [21] Z. Jiang, J. Araki, H. Ding, and G. Neubig. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977, 09 2021.
  - [22] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
  - [23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
  - [24] J. Li, J. Xu, S. Huang, Y. Chen, W. Li, J. Liu, Y. Lian, J. Pan, L. Ding, H. Zhou, et al. Large language model inference acceleration: A comprehensive hardware perspective. *arXiv preprint arXiv:2410.04466*, 2024.
  - [25] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In P. Gibbons, G. Pekhimenko, and C. D. Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024.
-

- [26] C. D. Manning. *Introduction to information retrieval*. Syngress Publishing,, 2008.
- [27] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [28] Ç. U. Öğdü, K. Arslanoğlu, and M. Karaköse. An adaptive multi-agent llm-based clinical decision support system integrating biomedical rag and web intelligence. *IEEE Access*, 2025.
- [29] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [30] J. Pennington, R. Socher, and C. D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- [31] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer. Deep contextualized word representations. In M. Walker, H. Ji, and A. Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.
- [32] W. A. Qader, M. M. Ameen, and B. I. Ahmed. An overview of bag of words; importance, implementation, applications, and challenges. In *2019 international engineering conference (IEC)*, pages 200–204. IEEE, 2019.
- [33] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [34] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- [35] S. T. Sreenivas, S. Muralidharan, R. Joshi, M. Chochowski, A. S. Mahabaleshwarkar, G. Shen, J. Zeng, Z. Chen, Y. Suhara, S. Diao, et al. Llm pruning and distillation in practice: The minitron approach. *arXiv preprint arXiv:2408.11796*, 2024.
- [36] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. *Advances in neural information processing systems*, 33:3008–3021, 2020.
- [37] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [38] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

- [39] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- [40] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [41] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pages 38087–38099. PMLR, 2023.
- [42] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian. Qa-lora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*, 2023.
- [43] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- [44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.